Foundations of Artificial Intelligence

AI-Based Automated API Testing Agent

A PROJECT REPORT

Submitted by

2116240701361 – NILO REXCY A



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report **AI-Based Automated API Testing Agent** is the bonafide work of **Nilo Rexcy A** who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
1	INTRODUCTION	6
	1.1 PROBLEM STATEMENT	7
	1.2 OBJECTIVES	8
	1.3 SCOPE OF THE PROJECT	9
2	LITERATURE REVIEW	10
	2.1 EXISTING SYSTEMS	11
	2.2 LIMITATIONS OF EXISTING APPROACHES	11
	2.3 RESEARCH GAP	12
3	PROPOSED SYSTEM	13
	3.1 ARCHITECTURE DIAGRAM	14
	3.2 WORKING PRINCIPLE	14
4	MODULES DESCRIPTION	16
	4.1 MODULE 1	16
	4.2 MODULE 2	18
5	IMPLEMENTATION AND RESULTS	20
	5.1 EXPERIMENTAL SETUP	20
	5.2 RESULTS	22
6	CONCLUSION AND FUTURE WORK	25
7	REFERENCES	27

ABSTRACT

The rapid growth of web applications, mobile applications, and cloud-based services has increased the importance of Application Programming Interfaces (APIs) in modern software development. APIs enable communication and data exchange between different software systems. Therefore, ensuring their reliability, accuracy, and functionality is essential for maintaining software quality. Traditional API testing methods require developers and software testers to manually prepare test cases, execute requests, verify responses, and identify defects. These activities are often repetitive, time-consuming, and susceptible to human errors.

The proposed project, AI-Based Automated API Testing Agent, aims to develop an intelligent web-based application that automates API testing using Artificial Intelligence algorithms without relying on Machine Learning techniques. The system uses rule-based reasoning, heuristic search, boundary value analysis, equivalence partitioning, and decision table testing to generate meaningful test cases and validate API functionality.

The application allows users to enter API endpoint details, configure HTTP request methods, provide request parameters, and initiate automated testing. Based on the supplied API specification and validation rules, the system automatically generates positive and negative test cases, including valid inputs, invalid values, missing parameters, boundary conditions, and incorrect data types. The generated requests are executed against the target API, and the responses are analyzed to identify potential functional defects.

The testing agent evaluates HTTP status codes, response structures, response times, and expected outputs. Test cases are classified as passed or failed according to

predefined validation rules. The system also produces testing reports containing execution summaries, identified defects, and recommendations for developers.

The proposed system aims to reduce manual testing effort, increase test coverage, improve defect identification, and simplify the API testing process. It demonstrates the practical application of symbolic Artificial Intelligence algorithms in software testing and provides a useful tool for developers, testers, and software quality assurance teams.

CHAPTER 1

INTRODUCTION

Application Programming Interfaces (APIs) play a significant role in modern software development. They act as communication interfaces between different software applications, allowing systems to exchange information and perform operations efficiently. APIs are widely used in e-commerce websites, banking applications, healthcare systems, educational platforms, mobile applications, and cloud services.

As software applications become increasingly interconnected, API reliability has become essential. An error in an API can result in incorrect information, application failures, unsuccessful transactions, or security vulnerabilities. Therefore, comprehensive API testing is necessary to ensure that software services operate according to their specified requirements.

Traditional API testing requires software testers to manually create HTTP requests, define input values, execute test scenarios, and verify the returned results. Although existing tools support API testing, preparing comprehensive test cases and interpreting the results can require considerable manual effort.

The AI-Based Automated API Testing Agent is proposed to simplify these activities through intelligent automation. The system uses Artificial Intelligence algorithms to generate test cases, execute API requests, analyze responses, and identify potential defects.

Unlike Machine Learning-based solutions, the proposed system uses symbolic Artificial Intelligence techniques such as rule-based reasoning, heuristic algorithms, decision tables, and boundary value analysis. These techniques enable the

application to make testing decisions based on explicitly defined rules and conditions without requiring model training or large datasets.

The system is intended to provide an accessible and practical solution for software developers, testers, and organizations that require efficient API testing.

1.1 PROBLEM STATEMENT

API testing is an essential part of software quality assurance. However, conventional testing approaches face several challenges.

Manual test-case generation requires testers to identify valid inputs, invalid inputs, boundary conditions, and possible error scenarios. When an API contains numerous parameters and endpoints, creating comprehensive test cases becomes difficult.

Additionally, manually executing and verifying API requests consumes time and increases the possibility of human errors. Certain edge cases, such as missing parameters, invalid data types, unexpected values, and incorrect response structures, may be overlooked.

Developers also need to analyze test results to determine whether an API behaves correctly. Without intelligent analysis, identifying potential defects and their causes can be challenging.

Therefore, an automated solution is required to generate meaningful API test cases, prioritize important scenarios, execute tests, detect potential defects, and produce understandable reports.

1.2 OBJECTIVES

The primary objective of the proposed project is to develop an AI-based automated API testing application using Artificial Intelligence algorithms without Machine Learning.

The specific objectives are:

1. To develop a user-friendly web interface for configuring and testing REST APIs.
2. To automatically generate positive and negative test cases using rule-based reasoning.
3. To implement boundary value analysis and equivalence partitioning for intelligent test generation.
4. To prioritize test scenarios using heuristic algorithms.
5. To execute API requests automatically using HTTP methods such as GET, POST, PUT, PATCH, and DELETE.
6. To validate API responses against predefined expected results.
7. To identify potential functional defects and input validation errors.
8. To generate detailed testing reports containing pass/fail results and defect information.
9. To reduce manual testing effort and improve the overall efficiency of API testing.

1.3 SCOPE OF THE PROJECT

The proposed system focuses on automated functional testing of REST APIs. It supports configurable API endpoints, HTTP methods, request headers, query parameters, and JSON request bodies.

The system is designed to generate test cases based on available API specifications and user-defined validation rules.

Testing includes valid input verification, invalid input handling, boundary value testing, required-field validation, data type verification, and HTTP response analysis.

The application is intended for software developers, software testing professionals, students, and quality assurance engineers.

The initial implementation focuses on functional testing and basic response-time measurement. Advanced security testing, large-scale load testing, and complete end-to-end workflow automation are considered possible future enhancements.

CHAPTER 2

LITERATURE REVIEW

Software testing is an important process in the Software Development Life Cycle (SDLC). It ensures that software applications operate according to specified functional and non-functional requirements.

API testing focuses on verifying the communication interfaces between software components. It involves sending requests to API endpoints and validating the responses.

Several existing tools provide capabilities for manual and automated API testing.

2.1 EXISTING SYSTEMS

POSTMAN

Postman is a widely used API development and testing platform. It enables developers to create HTTP requests, configure parameters, inspect responses, and organize automated testing collections.

Postman supports API assertions and test automation. However, users often need to configure test scripts or testing scenarios to achieve comprehensive validation.

APACHE JMETER

Apache JMeter is an open-source testing tool primarily used for evaluating application performance and load-handling capabilities.

It allows testers to simulate multiple requests and measure parameters such as response time, throughput, and error rates.

Although JMeter supports API testing, its primary strength is performance and load testing rather than automatic functional test-case generation.

SOAPUI

SoapUI is an API testing tool that supports SOAP and REST services. It provides functional testing, assertions, and data-driven testing capabilities.

However, test-case configuration and the identification of important edge cases may still require significant tester involvement.

2.2 LIMITATIONS OF EXISTING APPROACHES

Existing API testing approaches have several limitations.

First, manual test-case preparation requires time and technical knowledge. Testers need to identify input conditions, expected outputs, and possible failure scenarios.

Second, some tools require scripting or complex configuration to implement advanced validation.

Third, important boundary conditions and invalid input combinations may be overlooked when test scenarios are created manually.

Fourth, conventional execution results may not automatically explain the reason for a failed test case.

Finally, testing multiple API endpoints and repeatedly executing similar scenarios can increase testing effort.

These limitations create an opportunity for an intelligent API testing system that automatically generates and prioritizes meaningful test cases using explainable Artificial Intelligence algorithms.

2.3 RESEARCH GAP

Although API testing tools provide automation features, there is a need for a system that combines intelligent test-case generation, prioritization, execution, and defect reporting within a single accessible application.

The proposed system addresses this requirement by using symbolic Artificial Intelligence techniques rather than Machine Learning.

By applying predefined knowledge rules, constraints, and heuristic evaluation methods, the system can generate testing scenarios and make explainable decisions.

The approach aims to improve testing efficiency while remaining transparent, understandable, and suitable for practical software development environments.

CHAPTER 3

PROPOSED SYSTEM

The AI-Based Automated API Testing Agent is a web-based application designed to automate the process of API testing through Artificial Intelligence algorithms.

The system accepts API specifications, endpoint details, request parameters, and validation requirements from the user.

Based on this information, the intelligent testing engine analyzes the API inputs and automatically generates relevant test cases.

The generated test cases are prioritized according to their potential importance and executed against the target API.

The execution engine captures HTTP responses and forwards them to the validation module. The validation module compares actual responses against expected outcomes and identifies deviations.

Finally, the reporting module presents the testing results through a dashboard containing test summaries, identified defects, response-time information, and recommended corrective actions.

3.1 ARCHITECTURE DIAGRAM

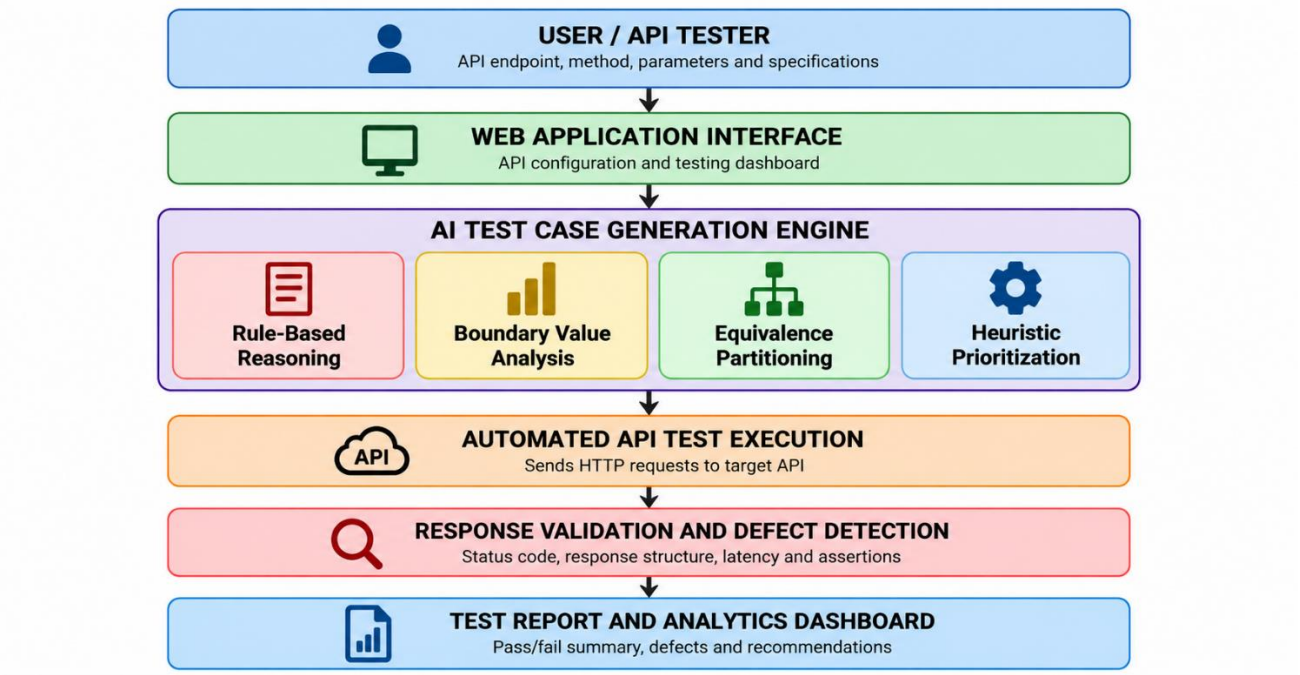


Figure 3.1: Architecture of the AI-Based Automated API Testing Agent

3.2 WORKING PRINCIPLE

The proposed system follows a systematic testing process.

Step 1: API Configuration

The user enters the API endpoint URL, selects an HTTP request method, and provides required headers, parameters, and request body details.

Step 2: Input Analysis

The system analyzes the API specification to identify required parameters, data types, valid input ranges, and expected response behavior.

Step 3: Intelligent Test Generation

The AI engine applies predefined rules, boundary value analysis, and equivalence partitioning to generate appropriate test cases.

Step 4: Test Case Prioritization

A heuristic algorithm assigns priority scores to test cases based on their potential risk and importance.

Step 5: Automated Execution

The system sends HTTP requests to the target API using the generated test inputs.

Step 6: Response Validation

The received responses are analyzed based on expected status codes, JSON structure, response values, and validation requirements.

Step 7: Defect Identification

If the actual API behavior differs from the specified expected behavior, the system identifies and records the failed test case.

Step 8: Report Generation

The application generates a detailed testing report containing execution results, pass/fail statistics, and information about identified defects.

CHAPTER 4

MODULES DESCRIPTION

The proposed system consists of two major modules that work together to automate API testing.

4.1 MODULE 1 – INTELLIGENT API TEST CASE GENERATION

The Intelligent API Test Case Generation Module is responsible for analyzing API requirements and automatically preparing test scenarios.

The module receives API configurations and validation constraints from the user.

Based on this information, the AI engine applies rule-based reasoning, boundary value analysis, equivalence partitioning, and heuristic prioritization.

The system generates positive and negative testing scenarios.

Positive test cases verify that the API accepts valid requests and produces the expected output.

Negative test cases verify that the API properly handles invalid values, missing fields, unsupported data types, and incorrect input formats.

The module also assigns priority scores to generated test cases so that important scenarios can be tested first.

Major Functions:

1. API endpoint configuration.
2. HTTP method selection.
3. Request parameter analysis.
4. Automatic test-case generation.

5. Positive and negative test-case preparation.
6. Boundary condition identification.
7. Heuristic-based test prioritization.

Example:

Consider an API endpoint that accepts an age parameter between 18 and 60.

Test Case	**Input**	**Expected Behavior**
TC01	Age = 25	Request accepted
TC02	Age = 18	Request accepted
TC03	Age = 60	Request accepted
TC04	Age = 17	Validation error
TC05	Age = 61	Validation error
TC06	Age = "abc"	Validation error
TC07	Age missing	Validation error

The expected behavior follows the supplied API contract. The module generates these scenarios to verify whether the API correctly enforces the defined requirements.

4.2 MODULE 2 – AUTOMATED TEST EXECUTION AND REPORT GENERATION

The Automated Test Execution and Report Generation Module is responsible for executing generated test cases and analyzing API responses.

The module sends HTTP requests using the input values generated by Module 1.

For every request, the system captures the response status code, response body, response time, and execution outcome.

The validation engine compares the actual response with the expected results defined by the API specification.

When a mismatch is detected, the system records the failure and provides a possible explanation.

For example, if a mandatory field is missing and the API accepts the request despite a requirement to reject it, the system identifies a potential input validation defect.

After executing all test cases, the module summarizes the results and generates a detailed testing report.

Major Functions:

1. Automated HTTP request execution.
2. API response collection.
3. HTTP status code verification.
4. Response body and JSON structure validation.
5. Response time measurement.
6. Defect identification.

7. Pass/fail classification.
8. Automated report generation.

Example Test Execution Results:

Test ID	Expected	Actual	Result
TC01	200 OK	200 OK	PASS
TC02	200 OK	200 OK	PASS
TC03	200 OK	200 OK	PASS
TC04	400 Bad Request	200 OK	FAIL
TC05	400 Bad Request	400 Bad Request	PASS

The table illustrates how the system identifies an unexpected response and records the corresponding failure.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 EXPERIMENTAL SETUP

The proposed AI-Based Automated API Testing Agent can be implemented using web development technologies and Python-based backend programming.

The frontend provides an interactive interface for entering API details, selecting testing options, and viewing results.

The backend manages test-case generation, API request execution, and response validation.

The Artificial Intelligence engine implements rule-based reasoning, boundary value analysis, equivalence partitioning, and heuristic test prioritization.

The system can be developed and tested using Visual Studio Code.

SOFTWARE REQUIREMENTS

Component	**Technology**
Frontend	HTML, CSS, JavaScript
Backend	Python, FastAPI
AI Algorithms	Rule-Based Reasoning, Heuristic Search

API Communication	HTTPX
Database	SQLite
Development Tool	Visual Studio Code
API Documentation	Swagger / OpenAPI
Operating System	Windows 10/11 or Linux

HARDWARE REQUIREMENTS

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Storage	1 GB available
Internet	Required for remote API testing
Display	Standard laptop or desktop

IMPLEMENTATION PROCEDURE

Phase 1: User Interface Development

A web interface is designed to allow users to enter API URLs, HTTP methods, headers, request parameters, and expected validation conditions.

Phase 2: AI Algorithm Implementation

Rule-based reasoning, boundary value analysis, and equivalence partitioning algorithms are implemented to automatically generate test scenarios.

Phase 3: Automated API Execution

The backend sends HTTP requests to configured API endpoints and collects response information.

Phase 4: Response Validation

The system compares actual responses with predefined expected outputs and identifies differences.

Phase 5: Result Visualization

The testing dashboard displays executed tests, passed and failed test cases, response times, and potential defects.

Phase 6: Report Generation

The system generates test reports that can be used by developers and testers for debugging and software quality improvement.

5.2 RESULTS

The effectiveness of the proposed system should be evaluated by executing tests against a sample REST API.

A sample application containing endpoints for user registration, user authentication, and profile management can be used for evaluation.

The system should generate test cases for different input conditions and execute them automatically.

The evaluation should measure the number of generated test cases, the proportion of successful tests, the number of identified defects, and the time required to complete testing.

SAMPLE RESULTS TABLE

The following values are illustrative only and must be replaced with actual test results obtained during implementation.

Parameter	**Illustrative Result**
Total Test Cases	30
Passed Test Cases	24
Failed Test Cases	6
Pass Percentage	80%
Fail Percentage	20%
Potential Defects Identified	6
Automated Report Generation	Successful

PERFORMANCE ANALYSIS

The system's testing efficiency can be evaluated based on execution duration and the number of test cases completed.

The percentage of successful test cases can be calculated using:

$$\text{Pass Percentage} = \frac{\text{Passed Tests}}{\text{Total Tests}} \times 100$$

For the illustrative results:

$$\text{Pass Percentage} = \frac{24}{30} \times 100 = 80\%$$

The failure percentage can be calculated using:

$$\text{Fail Percentage} = \frac{\text{Failed Tests}}{\text{Total Tests}} \times 100$$

For the illustrative results:

$$\text{Fail Percentage} = \frac{6}{30} \times 100 = 20\%$$

The system can also measure individual API response times to identify slow requests.

OBSERVATIONS

The following aspects should be examined during experimental evaluation:

1. Whether the AI engine generates appropriate test cases based on API specifications.
2. Whether the system detects known input validation defects.
3. Whether test-case prioritization executes higher-risk scenarios first.
4. Whether the response validation engine correctly identifies mismatches.

CHAPTER 6

CONCLUSION AND FUTURE WORK

CONCLUSION

The AI-Based Automated API Testing Agent presents a practical approach to improving software testing through Artificial Intelligence algorithms.

The proposed system automates test-case generation, API request execution, response validation, and defect reporting.

By applying rule-based reasoning, boundary value analysis, equivalence partitioning, and heuristic prioritization, the system demonstrates how intelligent testing decisions can be made without using Machine Learning.

The application is designed to reduce repetitive manual testing activities and help developers identify API defects more efficiently.

The proposed system also provides an understandable reporting mechanism that allows software testers to review testing outcomes and identify areas requiring improvement.

Overall, the project demonstrates the usefulness of symbolic Artificial Intelligence techniques in software quality assurance and provides a foundation for developing more advanced automated testing solutions.

FUTURE WORK

The proposed system can be enhanced through the following features:

1. Advanced API Security Testing: Implement additional checks for authentication, authorization, and common API security vulnerabilities.

2. API Workflow Testing: Automatically test interconnected API operations involving multiple endpoints.

3. Regression Testing: Compare current test results with previous executions to identify newly introduced defects.

4. Continuous Integration and Deployment: Integrate automated API testing into CI/CD pipelines.

5. Genetic Algorithm-Based Optimization: Use evolutionary algorithms to improve test-case selection and coverage.

6. Multi-API Testing Support: Extend the system to test multiple connected APIs within a single testing workflow.

7. Advanced Analytics Dashboard: Introduce detailed visualizations for test coverage, response time trends, and defect categories.

CHAPTER 7

REFERENCES

- Ammann, P. and Offutt, J. (2016), *Introduction to Software Testing*, 2nd Edition, Cambridge University Press.
- Apache Software Foundation (n.d.), *Apache JMeter User's Manual*. Available at: https://jmeter.apache.org/usermanual/
- Fielding, R.T. (2000), *Architectural Styles and the Design of Network-based Software Architectures*, Doctoral Dissertation, University of California, Irvine.
- OpenAPI Initiative (n.d.), *OpenAPI Specification*. Available at: https://spec.openapis.org/oas/latest.html
- OWASP Foundation (2023), *OWASP API Security Top 10 – 2023*. Available at: https://owasp.org/API-Security/
- Postman (n.d.), *Postman Learning Center*. Available at: https://learning.postman.com/
- Russell, S. and Norvig, P. (2021), *Artificial Intelligence: A Modern Approach*, 4th Edition, Pearson.